ContextStream MCP Server — Comprehensive Test Suite

> ***Usage:*** *Paste this entire prompt into a new Claude Code session with ContextStream MCP connected. Claude will execute all test cases, report pass/fail, and leave artifacts tagged* `[CS-TEST]` *for UI inspection.*

You are a QA engineer testing the ContextStream MCP server. Execute every test case below **in order**. For each test:

1. Run the MCP tool call exactly as described
2. Record the result: **PASS** (expected behavior) or **FAIL** (error, missing data, wrong response)
3. If a test returns an ID, save it — later tests may reference it
4. **Do NOT clean up** test artifacts — leave them for manual UI inspection
5. All created artifacts MUST use the `[CS-TEST]` prefix in their title for easy identification and later cleanup

After all suites complete, print a summary table with columns: `Suite | Test ID | Description | Result | Notes`.

Target workspace: sim-steward / simhub-plugin **Folder path:** `C:\Users\winth\dev\sim-steward\simhub-plugin`

Suite 1: Init & Context (3 tests)

T1.1 — Init session Call `init(folder_path="C:\\Users\\winth\\dev\\sim-steward\\simhub-plugin", client_name="claude")`.

- PASS if: response contains workspace name "sim-steward" and project "simhub-plugin"
- FAIL if: error, no workspace resolved, or wrong workspace

T1.2 — Context (fast mode) Call `context(user_message="ContextStream MCP test suite running", mode="fast")`.

- PASS if: returns without error
- FAIL if: error or timeout

T1.3 — Context (standard mode) Call `context(user_message="Testing context retrieval with lessons and preferences", mode="standard")`.

- PASS if: response includes any of: lessons, preferences, or contextstream rules
- FAIL if: empty response or error

Suite 2: Search (7 tests)

T2.1 — Keyword search Call `search(mode="keyword", query="DispatchAction", limit=3)`.

- PASS if: returns results with file paths ending in `.cs`
- FAIL if: 0 results or error

T2.2 — Pattern search (glob) Call `search(mode="pattern", query="*.csproj", limit=5)`.

- PASS if: returns file paths ending in `.csproj`
- FAIL if: 0 results or error

T2.3 — Semantic search Call `search(mode="semantic", query="incident detection and replay logic", limit=3)` .

- PASS if: returns results related to incident/replay code
- FAIL if: 0 results or error

T2.4 — Auto mode Call `search(mode="auto", query="WebSocket server binding", limit=3)` .

- PASS if: returns results (mode auto-selected) with file paths
- FAIL if: 0 results or error

T2.5 — Exhaustive search Call `search(mode="exhaustive", query="TODO", limit=10)` .

- PASS if: returns multiple results across different files
- FAIL if: 0 results or error

T2.6 — Search result file validation Take the first file path returned from T2.1. Use the `Read` tool to read that file at the returned line number.

- PASS if: file exists and contains the search term near the reported line
- FAIL if: file doesn't exist, or content doesn't match

T2.7 — Search with include_content Call `search(mode="keyword", query="class SimStewardPlugin", include_content=true, limit=1)` .

- PASS if: result includes actual code content (not just file paths)
- FAIL if: no content in response

Suite 3: Memory — Nodes (7 tests) ⚠️ KNOWN PAIN POINT

> ***Context:*** *Past sessions observed node IDs returned by* `search` *that 404'd on* `get_node` *. This suite specifically tests create→get round-trip integrity.*

T3.1 — Create node (fact) Call `memory(action="create_node", node_type="fact", title="[CS-TEST] Round-trip test node", content="This is a test fact created by the ContextStream MCP test suite. Timestamp: {current ISO time}")` .

- PASS if: returns a node ID (UUID format)
- FAIL if: error
- **Save the returned node_id as** `TEST_NODE_ID`

T3.2 — Get node (immediate round-trip) Call `memory(action="get_node", node_id=TEST_NODE_ID)` .

- PASS if: returns the node with matching title "[CS-TEST] Round-trip test node" and content containing "test fact"
- FAIL if: 404 error or content mismatch — **THIS IS THE CRITICAL REGRESSION TEST**

T3.3 — Update node Call `memory(action="update_node", node_id=TEST_NODE_ID, content="Updated content: round-trip verification successful. Updated at: {current ISO time}")` .

- PASS if: update acknowledged without error
- FAIL if: error

T3.4 — Get node (verify update persisted) Call `memory(action="get_node", node_id=TEST_NODE_ID)` .

- PASS if: content now contains "Updated content: round-trip verification successful"
- FAIL if: still shows old content or 404

T3.5 — List nodes (verify appears) Call `memory(action="list_nodes", node_type="fact")`.

- PASS if: test node with "[CS-TEST]" title appears in list
- FAIL if: not found in list despite successful creation

T3.6 — Supersede node Call `memory(action="supersede_node", node_id=TEST_NODE_ID, new_content="Superseded: this node was replaced during testing", reason="Test suite supersession")`.

- PASS if: returns new node ID and acknowledges supersession
- FAIL if: error
- **Save the returned new node_id as `TEST_NODE_ID_V2`**

T3.7 — Search → Get consistency check ⚠️ Call `memory(action="search", query="[CS-TEST] Round-trip test node")`. Then for EACH node ID returned, call `memory(action="get_node", node_id=<id>)`.

- PASS if: every ID returned by search resolves successfully via get_node
- FAIL if: any ID returns 404 — **this is the exact bug observed in past sessions**

Suite 4: Memory — Events (5 tests)

T4.1 — Create event Call `memory(action="create_event", event_type="note", title="[CS-TEST] Event round-trip test", content="Test event created by MCP test suite. Contains structured data:\n- key1: value1\n- key2: value2")`.

- PASS if: returns event ID
- FAIL if: error
- **Save as `TEST_EVENT_ID`**

T4.2 — Get event (round-trip) Call `memory(action="get_event", event_id=TEST_EVENT_ID)`.

- PASS if: returns event with matching title and content including "key1: value1"
- FAIL if: 404 or content mismatch

T4.3 — List events (verify appears) Call `memory(action="list_events")`.

- PASS if: test event with "[CS-TEST]" in title appears in the list
- FAIL if: not found

T4.4 — Update event Call `memory(action="update_event", event_id=TEST_EVENT_ID, title="[CS-TEST] Event round-trip test (UPDATED)", content="Updated event content with additional data:\n- key3: value3")`.

- PASS if: update acknowledged
- FAIL if: error

T4.5 — Get event (verify update) Call `memory(action="get_event", event_id=TEST_EVENT_ID)`.

- PASS if: title contains "(UPDATED)" and content contains "key3: value3"
- FAIL if: shows old content

Suite 5: Memory — Docs (7 tests) ⚠️ KNOWN PAIN POINT

> ***Context:*** *The user has experienced issues with documents not syncing between MCP creation and the ContextStream UI. This suite stress-tests doc CRUD and content fidelity.*

T5.1 — Create doc Call `memory(action="create_doc", doc_type="general", title="[CS-TEST] Doc Sync Verification", content="# Test Document\n\nThis document tests ContextStream doc sync fidelity.\n\n## Section 1: Basic Content\nParagraph with **bold**, *italic*, and code .\n\n## Section 2: Code Block\n```csharp\npublic class TestClass {\n public void TestMethod() {\n Console.WriteLine(\"Hello from test\");\n }\n}\n```\n\n## Section 3: Table\n| Column A | Column B |\n|----------|----------|\n| val1 | val2 |\n\n## Section 4: List\n- Item 1\n- Item 2\n - Nested item\n- Item 3")`.

- PASS if: returns doc ID
- FAIL if: error
- **Save as** `TEST_DOC_ID`

T5.2 — Get doc by ID (content fidelity) Call `memory(action="get_doc", doc_id=TEST_DOC_ID)`.

- PASS if: ALL of these are true:
  - Title matches "[CS-TEST] Doc Sync Verification"
  - Content contains the code block with `public class TestClass`
  - Content contains the table with "Column A" and "Column B"
  - Content contains the nested list item
- FAIL if: any content is truncated, mangled, or missing — **this is the sync regression test**

T5.3 — Get doc by title query Call `memory(action="get_doc", doc_id="Doc Sync Verification")`.

- PASS if: returns the same doc as T5.2 (matching ID)
- FAIL if: not found or returns a different doc

T5.4 — Update doc (substantial change) Call `memory(action="update_doc", doc_id=TEST_DOC_ID, content="# Test Document (UPDATED)\n\nOriginal content replaced with updated version.\n\n## New Section\nThis section was added during the update test.\n\n## Preserved Formatting\n```python\ndef test():\n return 'updated'\n```\n\n| New Col A | New Col B | New Col C |\n|-----------|-----------|----------|\n| x | y | z |")`.

- PASS if: update acknowledged
- FAIL if: error

T5.5 — Get doc (verify update persisted) Call `memory(action="get_doc", doc_id=TEST_DOC_ID)`.

- PASS if: content contains "(UPDATED)" in title and "New Section" and the python code block
- FAIL if: still shows old content — **indicates sync/cache issue**

T5.6 — List docs (verify appears) Call `memory(action="list_docs")`.

- PASS if: test doc with "[CS-TEST]" title appears
- FAIL if: not found in listing

T5.7 — Large content doc ⚠️ Create a doc with >2000 characters of content: Call `memory(action="create_doc", doc_type="general", title="[CS-TEST] Large Content Stress Test", content="{generate a ~2500 char markdown document with multiple sections, code blocks, tables, and lists}")`. Then immediately `get_doc` by the returned ID.

- PASS if: returned content length matches what was sent (within 5% tolerance)
- FAIL if: content is truncated — **regression test for large doc sync**
- **Save as** `TEST_LARGE_DOC_ID`

Suite 6: Memory — Tasks (5 tests)

T6.1 — Create task Call `memory(action="create_task", title="[CS-TEST] Task round-trip test", priority="medium", description="Test task for MCP verification")`.

- PASS if: returns task ID
- FAIL if: error
- **Save as** `TEST_TASK_ID`

T6.2 — Get task (round-trip) Call `memory(action="get_task", task_id=TEST_TASK_ID)`.

- PASS if: returns task with matching title and priority "medium"
- FAIL if: 404 or mismatch

T6.3 — Update task status progression Call `memory(action="update_task", task_id=TEST_TASK_ID, task_status="in_progress")`. Then call `memory(action="update_task", task_id=TEST_TASK_ID, task_status="completed")`.

- PASS if: both updates succeed, final status is "completed"
- FAIL if: either update fails

T6.4 — List tasks (verify appears) Call `memory(action="list_tasks")`.

- PASS if: test task appears in list
- FAIL if: not found

T6.5 — Create task with plan linkage Call `memory(action="create_task", title="[CS-TEST] Plan-linked task", priority="low", description="Task linked to test plan")`.

- PASS if: returns task ID (plan linkage optional — just verify task creation works standalone)
- FAIL if: error
- **Save as** `TEST_PLAN_TASK_ID`

Suite 7: Memory — Todos (5 tests)

T7.1 — Create todo Call `memory(action="create_todo", title="[CS-TEST] Todo round-trip test", todo_priority="medium")`.

- PASS if: returns todo ID
- FAIL if: error
- **Save as** `TEST_TODO_ID`

T7.2 — Get todo (round-trip) Call `memory(action="get_todo", todo_id=TEST_TODO_ID)`.

- PASS if: returns todo with matching title and priority
- FAIL if: 404 or mismatch

T7.3 — Update todo Call `memory(action="update_todo", todo_id=TEST_TODO_ID, todo_priority="high")`. Then `get_todo` to verify.

- PASS if: priority updated to "high"
- FAIL if: still "medium" or error

T7.4 — Complete todo Call `memory(action="complete_todo", todo_id=TEST_TODO_ID)`. Then `get_todo` to verify status.

- PASS if: status is "completed"
- FAIL if: still "pending" or error

T7.5 — List todos Call `memory(action="list_todos")` .

- PASS if: test todo appears in list
- FAIL if: not found

Suite 8: Memory — Diagrams (5 tests)

T8.1 — Create diagram Call `memory(action="create_diagram", diagram_type="flowchart", title="[CS-TEST] Diagram round-trip test", content="graph TD\n A[Start] --> B{Decision}\n B -->|Yes| C[Action 1]\n B -->|No| D[Action 2]\n C --> E[End]\n D --> E")` .

- PASS if: returns diagram ID
- FAIL if: error
- **Save as** `TEST_DIAGRAM_ID`

T8.2 — Get diagram (round-trip) Call `memory(action="get_diagram", diagram_id=TEST_DIAGRAM_ID)` .

- PASS if: returns diagram with mermaid content containing "graph TD" and "Decision"
- FAIL if: 404 or content missing

T8.3 — Update diagram Call `memory(action="update_diagram", diagram_id=TEST_DIAGRAM_ID, content="graph LR\n A[Updated Start] --> B[Updated End]")` .

- PASS if: update acknowledged
- FAIL if: error

T8.4 — Get diagram (verify update) Call `memory(action="get_diagram", diagram_id=TEST_DIAGRAM_ID)` .

- PASS if: content now contains "graph LR" and "Updated Start"
- FAIL if: still shows old content

T8.5 — List diagrams Call `memory(action="list_diagrams")` .

- PASS if: test diagram appears
- FAIL if: not found

Suite 9: Session Operations (8 tests)

T9.1 — Capture decision Call `session(action="capture", event_type="decision", title="[CS-TEST] Test decision capture", content="Decision: Use MCP-only verification for test suite. Rationale: Faster and fully autonomous.", importance="low")` .

- PASS if: capture acknowledged
- FAIL if: error

T9.2 — Capture lesson Call `session(action="capture_lesson", title="[CS-TEST] Test lesson", trigger="Running test suite", impact="Validates MCP server reliability", prevention="Regular test runs", severity="low", keywords=["testing", "cs-test"])` .

- PASS if: lesson captured
- FAIL if: error

T9.3 — Get lessons (verify) Call `session(action="get_lessons")` .

- PASS if: returns lessons (list may or may not include the just-created one depending on indexing)
- FAIL if: error

T9.4 — Remember Call `session(action="remember", content="[CS-TEST] The MCP test suite was last run at {current ISO time}")`.

- PASS if: acknowledged
- FAIL if: error

T9.5 — Recall Call `session(action="recall", query="[CS-TEST] MCP test suite last run")`.

- PASS if: returns results (may include the remembered fact)
- FAIL if: error

T9.6 — Capture plan Call `session(action="capture_plan", title="[CS-TEST] Test plan", steps=["Step 1: Initialize", "Step 2: Create artifacts", "Step 3: Verify round-trips", "Step 4: Report results"])`.

- PASS if: returns plan ID
- FAIL if: error
- **Save as `TEST_PLAN_ID`**

T9.7 — Get plan Call `session(action="get_plan", plan_id=TEST_PLAN_ID)`.

- PASS if: returns plan with 4 steps matching what was created
- FAIL if: 404 or steps missing

T9.8 — List plans Call `session(action="list_plans")`.

- PASS if: test plan with "[CS-TEST]" title appears in list
- FAIL if: not found

Suite 10: Project Operations (6 tests)

T10.1 — List projects Call `project(action="list")`.

- PASS if: returns at least one project
- FAIL if: empty or error

T10.2 — Get project Call `project(action="get")`.

- PASS if: returns project details for simhub-plugin
- FAIL if: error

T10.3 — Index status Call `project(action="index_status")`.

- PASS if: returns status (ready, indexing, or stale)
- FAIL if: error

T10.4 — Statistics Call `project(action="statistics")`.

- PASS if: returns statistics with file counts or similar metrics
- FAIL if: error

T10.5 — Files Call `project(action="files", page_size=5)`.

- PASS if: returns file list with paths
- FAIL if: error or empty

T10.6 — Recent changes Call `project(action="recent_changes", limit=3)`.

- PASS if: returns recent git commits
- FAIL if: error

Suite 11: Workspace Operations (3 tests)

T11.1 — List workspaces Call `workspace(action="list")`.

- PASS if: returns at least one workspace including "sim-steward"
- FAIL if: empty or error

T11.2 — Get workspace Call `workspace(action="get")`.

- PASS if: returns workspace details
- FAIL if: error

T11.3 — Index settings Call `workspace(action="index_settings")`.

- PASS if: returns settings (or permission error if not admin — note which)
- FAIL if: unexpected error

Suite 12: Graph Operations (4 tests)

T12.1 — Dependencies Call `graph(action="dependencies", target_id="SimStewardPlugin.cs", target_type="module")`.

- PASS if: returns dependency data (even if empty for this file)
- FAIL if: unexpected error (not a "no data" response)

T12.2 — Impact analysis Call `graph(action="impact", target_id="DispatchAction", target_type="function", change_type="modify_signature")`.

- PASS if: returns impact analysis results
- FAIL if: error

T12.3 — Usages Call `graph(action="usages", target_id="IRSDKSharper")`.

- PASS if: returns files/components that use IRSDKSharper
- FAIL if: error

T12.4 — Circular dependencies Call `graph(action="circular_dependencies", limit=5)`.

- PASS if: returns results (empty list is OK — means no circular deps)
- FAIL if: error

Suite 13: Skills (4 tests) ⚠️ KNOWN PAIN POINT

> ***Context:*** *`skill(import, format=auto)` has been known to split multi-section files into many personal skills. This suite tests safe CRUD.*

T13.1 — List skills Call `skill(action="list")`.

- PASS if: returns skill list including known skills (e.g., "simsteward-deploy", "contextstream")
- FAIL if: empty or error

T13.2 — Get skill Pick the first skill ID from T13.1. Call `skill(action="get", skill_id=<id>)`.

- PASS if: returns skill with instruction_body present
- FAIL if: missing instruction_body or error

T13.3 — Create skill Call `skill(action="create", name="cs-test-skill", title="[CS-TEST] Test Skill", description="Skill created by MCP test suite", instruction_body="# Test Skill\n\nThis skill does nothing. It exists to verify skill CRUD.\n\n## Steps\n1. Do nothing\n2. Report success", categories=["testing", "cs-test"], status="draft")`.

- PASS if: returns skill ID
- FAIL if: error
- **Save as** `TEST_SKILL_ID`

T13.4 — Get created skill (round-trip) Call `skill(action="get", skill_id=TEST_SKILL_ID)`.

- PASS if: returns skill with title "[CS-TEST] Test Skill" and instruction_body containing "This skill does nothing"
- FAIL if: 404, wrong content, or instruction_body missing

Suite 14: Reminders (4 tests)

T14.1 — Create reminder Call `reminder(action="create", title="[CS-TEST] Test reminder", content="Reminder created by MCP test suite", priority="low", remind_at="{ISO time 1 hour from now}")`.

- PASS if: returns reminder ID
- FAIL if: error
- **Save as** `TEST_REMINDER_ID`

T14.2 — List reminders Call `reminder(action="list")`.

- PASS if: test reminder appears in list
- FAIL if: not found

T14.3 — Active reminders Call `reminder(action="active")`.

- PASS if: returns list (test reminder may or may not appear depending on remind_at timing)
- FAIL if: error

T14.4 — Complete reminder Call `reminder(action="complete", reminder_id=TEST_REMINDER_ID)`.

- PASS if: completion acknowledged
- FAIL if: error

Suite 15: Instruct / RAM (4 tests)

> ***Note:*** *These require a session_id. Use* `"cs-test-session"` *as the session_id.*

T15.1 — Push entries Call `instruct(action="push", session_id="cs-test-session", entries=[{"text": "[CS-TEST] Instruction 1: Always verify round-trips", "source": "test-suite"}, {"text": "[CS-TEST] Instruction 2: Check for 404s on get_node", "source": "test-suite", "critical": true}])`.

- PASS if: push acknowledged
- FAIL if: error

T15.2 — Get entries Call `instruct(action="get", session_id="cs-test-session")`.

- PASS if: returns the 2 pushed entries
- FAIL if: empty or error

T15.3 — Stats Call `instruct(action="stats", session_id="cs-test-session")`.

- PASS if: returns cache statistics
- FAIL if: error

T15.4 — Ack entries Take the entry IDs from T15.2. Call `instruct(action="ack", session_id="cs-test-session", ids=[<id1>, <id2>])`.

- PASS if: acknowledgment succeeded
- FAIL if: error

Suite 16: Integration Status (2 tests)

T16.1 — All integrations status Call `integration(provider="all", action="status")`.

- PASS if: returns status for available integrations (connected or not)
- FAIL if: unexpected error

T16.2 — GitHub integration search (if connected) If T16.1 shows GitHub connected, call `integration(provider="github", action="repos")`.

- PASS if: returns repo list
- FAIL if: error (SKIP if GitHub not connected)

Suite 17: Help & Utility (4 tests)

T17.1 — List tools Call `help(action="tools")`.

- PASS if: returns tool list with 10+ tools
- FAIL if: empty or error

T17.2 — Auth check Call `help(action="auth")`.

- PASS if: returns current user info
- FAIL if: error

T17.3 — Version Call `help(action="version")`.

- PASS if: returns version string
- FAIL if: error

T17.4 — Team status Call `help(action="team_status")`.

- PASS if: returns team/subscription info (or "no team" — both valid)
- FAIL if: unexpected error

Suite 18: Stress & Edge Cases (6 tests) ⚠️

T18.1 — Unicode in titles/content Call `memory(action="create_node", node_type="fact", title="[CS-TEST] Unicode: 日本語テスト 🚀 émojis & spëcial «chars»", content="Content with unicode: ñ, ü, ø,`

`中文, العربية, backticks: \` code`, pipes: |col1|col2|")`.

- PASS if: returns ID, and `get_node` returns content with all unicode preserved
- FAIL if: content mangled, truncated, or error
- **Save as `TEST_UNICODE_NODE_ID`**

T18.2 — Rapid sequential creates Create 3 nodes in rapid succession (no delay between calls):

1. `memory(action="create_node", node_type="fact", title="[CS-TEST] Rapid 1")`
2. `memory(action="create_node", node_type="fact", title="[CS-TEST] Rapid 2")`
3. `memory(action="create_node", node_type="fact", title="[CS-TEST] Rapid 3")` Then `list_nodes` and verify all 3 appear.

- PASS if: all 3 created and all 3 retrievable via `get_node`
- FAIL if: any creation fails, any ID returns 404, or any missing from list

T18.3 — Invalid UUID handling Call `memory(action="get_node", node_id="00000000-0000-0000-0000-000000000000")`.

- PASS if: returns a clean 404 "not found" error
- FAIL if: crashes, hangs, or returns unexpected data

T18.4 — Search immediately after create Call `memory(action="create_node", node_type="fact", title="[CS-TEST] Immediate search target XYZ789", content="Unique content for immediate indexing test: QRS456")`. Then immediately call `memory(action="search", query="XYZ789 QRS456")`.

- PASS if: the just-created node appears in search results
- FAIL if: not found — **indicates indexing lag** (note the delay if any)

T18.5 — Empty content handling Call `memory(action="create_node", node_type="fact", title="[CS-TEST] Empty content test", content="")`.

- PASS if: creates successfully (empty content allowed) or returns clear validation error
- FAIL if: crashes or ambiguous error

T18.6 — Memory search → get consistency ⚠️ CRITICAL Call `memory(action="search", query="[CS-TEST]")`. For EVERY node ID returned in the search results, call `memory(action="get_node", node_id=<id>)`.

- PASS if: 100% of search-returned IDs resolve via get_node
- FAIL if: ANY ID returns 404 — **this is the primary regression test for the known node ghost bug**
- Report: "{N} of {M} IDs resolved successfully"

Final Report

After completing all suites, produce this output:

Summary Table

```
| Suite | Test | Description                                | Result | Notes |
|-------|------|--------------------------------------------|--------|-------|
| 1     | T1.1 | Init session                               | ?      |       |
| 1     | T1.2 | Context (fast)                             | ?      |       |
| ...   | ...  | ...                                        | ...    | ...   |
```

Statistics

- Total tests: {N}
- Passed: {N}
- Failed: {N}
- Skipped: {N}

Known Pain Point Results

Highlight these specifically:

1. **Node 404 ghost bug (T3.7, T18.6):** PASS/FAIL — {details}
2. **Doc sync fidelity (T5.2, T5.5, T5.7):** PASS/FAIL — {details}
3. **Skill CRUD (T13.3, T13.4):** PASS/FAIL — {details}
4. **Search→Get consistency (T18.6):** PASS/FAIL — {N}/{M} resolved
5. **Immediate indexing (T18.4):** PASS/FAIL — {details}

Artifacts Left for UI Inspection

List all `[CS-TEST]` artifacts with their IDs and types:

```
| Type     | Title                                    | ID   |
|----------|------------------------------------------|------|
| node     | [CS-TEST] Round-trip test node           | ...  |
| doc      | [CS-TEST] Doc Sync Verification          | ...  |
| ...      | ...                                      | ...  |
```

To clean up: In a future session, search for `[CS-TEST]` and delete all matching artifacts.